ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)»

Отчет по лабораторной работе №2 по предмету «Операционные системы и архитектура компьютера»

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра ЦОП ВО «ТОП-ИТ» Учебная группа М8О-205БВ-25

ФИО обучающегося Изотов Арсений Антонович

15 октября

2026г.

дата

Москва

2026

ВВЕДЕНИЕ

Цель работы: приобретение практических навыков управления процессами в ОС и обеспечения обмена данных между процессами посредством каналов (pipe).

Задача: разработать программу на языке C, состоящую из двух независимых исполняемых модулей - родительского и дочернего процессов, - которые взаимодействуют через безымянный канал (pipe) с перенаправлением стандартных потоков ввода/вывода дочернего процесса.

Вариант: 9.

Постановка задачи (вариант 9):

1. Родительский процесс запрашивает у пользователя имя файла и открывает его на чтение.
2. Родительский процесс создаёт дочерний процесс и перенаправляет:
    - stdin дочернего процесса - в открытый файл;
    - stdout и stderr дочернего процесса - в pipe1.
3. Родительский процесс читает из pipe1 и выводит полученные данные в свой стандартный поток вывода.
4. Дочерний процесс читает команды вида число число число<endline>, делит первое число на все последующие и печатает результат.
5. При делении на 0 дочерний процесс завершается с кодом ошибки, родительский — вслед за ним.
6. Родительский и дочерний процессы представлены разными программами.

ОС: Windows. API**:** Win32. Среда сборки: CMake + MinGW.

АРХИТЕКТУРА РЕШЕНИЯ

Проект состоит из двух независимых исполняемых модулей и одного общего модуля утилит(табл.1).

Файл	Назначение
src/parent.c	Родительский процесс: запрос имени файла, создание pipe, запуск child.exe, чтение из pipe, вывод результатов
src/child.c	Дочерний процесс: чтение команд из stdin (файла), деление чисел, вывод в stdout (pipe)
src/common.c / src/common.h	Утилита печати системных ошибок WinAPI через FormatMessage

Таблица 1 – архитектура проекта

Схема решения представлена на рисунке 1:

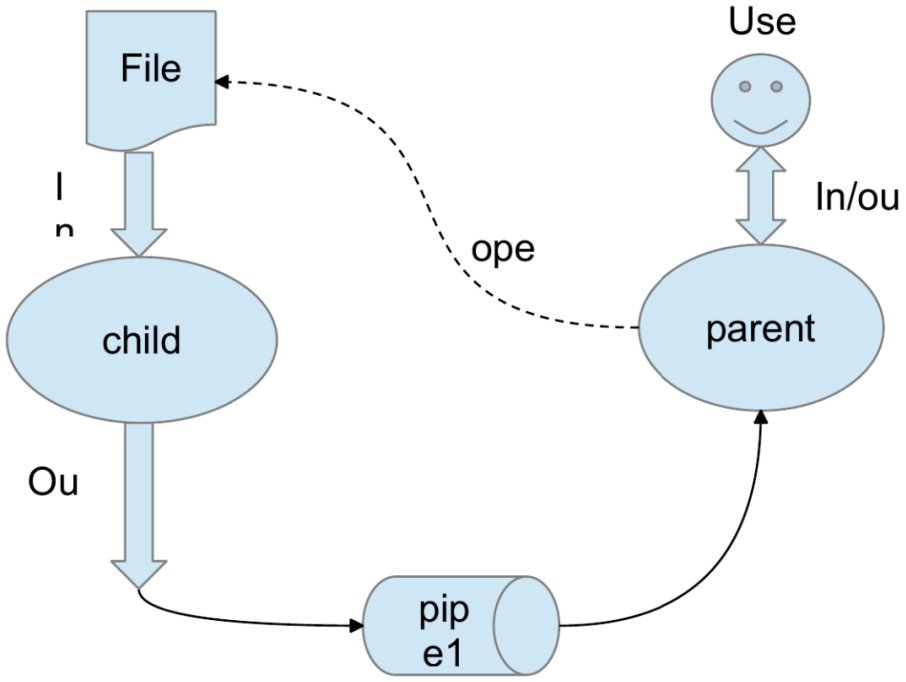


Рисунок 1 – схема решения

КЛЮЧЕВЫЕ МОМЕНТЫ КОДА

1) Создание наследуемого файла, pipe и перенаправление потоков дочернего процесса. Код представлен на рисунке 2.

```
SECURITY_ATTRIBUTES sa = { sizeof(sa), NULL, TRUE };
HANDLE hFile = CreateFileA(filename, GENERIC_READ, FILE_SHARE_READ,
                           &sa, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile == INVALID_HANDLE_VALUE) {
    print_error("CreateFileA(input)");
    return 1;
}

HANDLE hRead = NULL, hWrite = NULL;
if (!CreatePipe(&hRead, &hWrite, &sa, 0)) {
    print_error("CreatePipe");
    close_if_valid(hFile);
    return 1;
}

if (!SetHandleInformation(hRead, HANDLE_FLAG_INHERIT, 0)) {
    print_error("SetHandleInformation");
    close_if_valid(hRead);
    close_if_valid(hWrite);
    close_if_valid(hFile);
    return 1;
}

STARTUPINFOA si;
ZeroMemory(&si, sizeof(si));
si.cb = sizeof(si);
si.dwFlags = STARTF_USESTDHANDLES;
si.hStdInput  = hFile;
si.hStdOutput = hWrite;
si.hStdError  = hWrite;
```

Рисунок 2 – инициализация IPC в родителе

2) Запуск дочернего процесса и чтение из pipe. Код на рисунках 3,4 и 5.

```
PROCESS_INFORMATION pi;
ZeroMemory(&pi, sizeof(pi));

if (!CreateProcessA(NULL, "child.exe", NULL, NULL, TRUE,
                    0, NULL, NULL, &si, &pi)) {
    print_error("CreateProcessA");
    close_if_valid(hRead);
    close_if_valid(hWrite);
    close_if_valid(hFile);
    return 1;
}
```

Рисунок 3 – запуск дочернего процесса

```
CloseHandle(hWrite);
CloseHandle(hFile);

char buf[4096];
DWORD n;
BOOL ok = TRUE;
while (ok) {
    ok = ReadFile(hRead, buf, sizeof(buf), &n, NULL);
    if (!ok) {
        DWORD err = GetLastError();
        if (err == ERROR_BROKEN_PIPE) break;
        print_error("ReadFile");
        break;
    }
    if (n == 0) break;
    if (fwrite(buf, 1, n, stdout) != n) {
        perror("fwrite");
        break;
    }
}
CloseHandle(hRead);
```

Рисунок 4 – чтение файла

```
DWORD wait = WaitForSingleObject(pi.hProcess, INFINITE);
if (wait == WAIT_FAILED) {
    print_error("WaitForSingleObject");
}

DWORD code = 0;
if (!GetExitCodeProcess(pi.hProcess, &code)) {
    print_error("GetExitCodeProcess");
}

CloseHandle(pi.hProcess);
CloseHandle(pi.hThread);

if (code != 0) {
    fprintf(stderr, "child terminated with code %lu\n", code);
    return 1;
}
return 0;
```

Рисунок 5 – получение ответа от дочернего процесса

3) Решение варианта 9, подсчет которого производится в дочернем процессе представлено на рисунках 6 и 7.

```c
while (fgets(line, sizeof(line), stdin)) {
    const char *p = line;
    float acc;
    int off = 0;

    if (sscanf(p, "%f%n", &acc, &off) != 1) {
        continue;
    }
    p += off;

    int div_by_zero = 0;
    for (;;) {
        float b;
        int n = 0;
        if (sscanf(p, "%f%n", &b, &n) != 1) break;
        p += n;

        if (b == 0.0f) { div_by_zero = 1; break; }
        acc /= b;
    }
```

Рисунок 6 – вариант 9 (начало)

```c
    if (div_by_zero) {
        fprintf(stderr, "division by zero\n");
        return 1;
    }

    if (printf("%g\n", acc) < 0) {
        perror("printf");
        return 2;
    }
    if (fflush(stdout) != 0) {
        perror("fflush");
        return 2;
    }
}

if (ferror(stdin)) {
    perror("fgets");
    return 2;
}
return 0;
```

Рисунок 7 – вариант 9 (конец)

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были освоены механизмы WinAPI для управления процессами и организации межпроцессного взаимодействия. Реализована программа из двух независимых исполняемых модулей: родительский процесс открывает заданный пользователем файл, создаёт безымянный канал (CreatePipe) и запускает дочерний процесс (CreateProcessA) с перенаправлением стандартных потоков через структуру STARTUPINFO. Дочерний процесс читает команды из файла, выполняет деление чисел и передаёт результат родителю через pipe, который выводит его в свою консоль. Отработаны ключевые аспекты системного программирования: наследование хендлов через SECURITY_ATTRIBUTES, снятие флага наследования с read-конца канала, корректный порядок закрытия дескрипторов для предотвращения взаимоблокировок.

Вывод: системные примитивы WinAPI CreatePipe, CreateProcessA и STARTUPINFO предоставляют полноценную замену POSIX-связке pipe/fork/dup2 для организации межпроцессного взаимодействия в Windows. Полученные навыки работы с наследованием хендлов, перенаправлением потоков ввода-вывода и синхронизацией процессов являются базовыми для построения надёжных многопроцессных приложений.